

ใบงานสัปดาห์ที่ 4

SQL พื้นฐาน

การสร้างและจัดการฐานข้อมูล Student Management

ตารางที่ใช้: **students** และ **courses**

ชื่อ-สกุล ปกรณ์ ทองเจริญ

รหัสนักศึกษา 6811011662008

เนื้อหาในรายงาน

ส่วนที่ 1 SQL Script · ส่วนที่ 2 ผลลัพธ์การทำงาน · ส่วนที่ 3 อธิบายคำสั่ง SQL

ส่วนที่ 1 SQL Script

ไฟล์ `student_management_mysql.sql` (ใช้กับ MySQL / MariaDB / phpMyAdmin / XAMPP)

```
1  -- =====
2  -- ใบงานสัปดาห์ที่ 4 เรื่อง : SQL พื้นฐาน
3  -- ฐานข้อมูล : Student Management (สำหรับ MySQL / MariaDB / phpMyAdmin / XAMPP)
4  -- ตาราง      : students , courses
5  -- ผู้จัดทำ   : ปภังกร ทองเจริญ รหัสนักศึกษา 6811011662008
6  -- =====
7
8
9  -- =====
10 -- ส่วนที่ 0 : สร้างฐานข้อมูลและเลือกใช้งาน
11 -- =====
12 DROP DATABASE IF EXISTS student_management;
13 CREATE DATABASE student_management
14     DEFAULT CHARACTER SET utf8mb4
15     DEFAULT COLLATE utf8mb4_unicode_ci;
16
17 USE student_management;
18
19
20 -- =====
21 -- ส่วนที่ 1 : สร้างตารางด้วยคำสั่ง CREATE TABLE
22 -- =====
23
24 -- ----- ตารางที่ 1 : courses (รายวิชา) -----
25 -- สร้างตาราง courses ก่อน เพราะตาราง students ต้องอ้างอิง (FOREIGN KEY) มาที่ตารางนี้
26 CREATE TABLE courses (
27     course_id INT AUTO_INCREMENT PRIMARY KEY, -- รหัสวิชา (คีย์หลัก เพิ่มค่าอัตโนมัติ)
28     course_code VARCHAR(10) NOT NULL UNIQUE, -- รหัสวิชาที่ใช้เรียก เช่น CS101 (ห้ามซ้ำ)
29     course_name VARCHAR(100) NOT NULL, -- ชื่อวิชา (ห้ามเป็นค่าว่าง)
30     credits INT NOT NULL DEFAULT 3, -- หน่วยกิต ถ้าไม่ระบุใช้ค่า 3
31     instructor VARCHAR(60), -- ชื่ออาจารย์ผู้สอน
32     semester VARCHAR(10), -- ภาคเรียน เช่น 1/2568
33     CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 6) -- หน่วยกิตต้องอยู่ระหว่าง 1-6
34 ) ENGINE=InnoDB;
35
36 -- ----- ตารางที่ 2 : students (นักศึกษา) -----
37 CREATE TABLE students (
38     student_id INT AUTO_INCREMENT PRIMARY KEY, -- รหัสอ้างอิงภายใน (คีย์หลัก)
39     student_code VARCHAR(12) NOT NULL UNIQUE, -- รหัสนักศึกษา เช่น 6712001 (ห้ามซ้ำ)
40     first_name VARCHAR(50) NOT NULL, -- ชื่อ (ห้ามเป็นค่าว่าง)
41     last_name VARCHAR(50) NOT NULL, -- นามสกุล (ห้ามเป็นค่าว่าง)
42     gender VARCHAR(10), -- เพศ
43     age INT NOT NULL, -- อายุ
44     email VARCHAR(80) UNIQUE, -- อีเมล (ห้ามซ้ำ)
45     gpa DECIMAL(3,2) DEFAULT 0.00, -- เกรดเฉลี่ย ทศนิยม 2 ตำแหน่ง
46     enroll_date DATE, -- วันที่เข้าศึกษา
47     course_id INT, -- วิชาที่ลงทะเบียน (คีย์นอก)
48     CONSTRAINT chk_age CHECK (age >= 15), -- อายุต้องไม่น้อยกว่า 15 ปี
49     CONSTRAINT fk_student_course FOREIGN KEY (course_id)
50         REFERENCES courses(course_id)
51         ON UPDATE CASCADE
52         ON DELETE SET NULL -- ถ้าลบวิชา ค่าในตารางนักศึกษาจะเป็น NULL
53 ) ENGINE=InnoDB;
54
55
56 -- =====
57 -- ส่วนที่ 2 : เพิ่มข้อมูลด้วยคำสั่ง INSERT
58 -- =====
59
60 -- ----- 2.1 เพิ่มข้อมูลรายวิชา (เพิ่มหลายแถวในคำสั่งเดียว) -----
61 INSERT INTO courses (course_code, course_name, credits, instructor, semester) VALUES
62 ('CS101', 'Introduction to Programming', 3, 'อ.สมชาย ใจดี', '1/2568'),
```

```

63 ('CS102', 'Database Systems', 3, 'อ.สมหญิง รักเรียน', '1/2568'),
64 ('CS201', 'Web Development', 3, 'อ.วิชัย เก่งกาจ', '1/2568'),
65 ('MA101', 'Mathematics for Computing', 3, 'อ.มานี มีวิชา', '1/2568'),
66 ('EN101', 'English for IT', 2, 'อ. John Smith', '1/2568');
67
68 -- ----- 2.2 เพิ่มข้อมูลนักศึกษา -----
69 INSERT INTO students
70 (student_code, first_name, last_name, gender, age, email, gpa, enroll_date, course_id)
71 VALUES
72 ('6712001', 'สมชาย', 'ใจงาม', 'ชาย', 20, 'somchai@dusit.ac.th', 3.25, '2025-06-01', 1),
73 ('6712002', 'สมหญิง', 'รักดี', 'หญิง', 19, 'somying@dusit.ac.th', 3.75, '2025-06-01', 2),
74 ('6712003', 'ปิติ', 'ยินดี', 'ชาย', 22, 'piti@dusit.ac.th', 2.85, '2025-06-01', 1),
75 ('6712004', 'มานี', 'มีนา', 'หญิง', 21, 'manee@dusit.ac.th', 3.50, '2025-06-02', 3),
76 ('6712005', 'วีระ', 'กล้าหาญ', 'ชาย', 23, 'weera@dusit.ac.th', 2.40, '2025-06-02', 2),
77 ('6712006', 'ชูใจ', 'ใจดี', 'หญิง', 18, 'choojai@dusit.ac.th', 3.90, '2025-06-03', 4),
78 ('6712007', 'อนุชา', 'พากเพียร', 'ชาย', 20, 'anucha@dusit.ac.th', 3.10, '2025-06-03', 3),
79 ('6712008', 'กาญจนา', 'ศรีสุข', 'หญิง', 24, 'kanjana@dusit.ac.th', 2.95, '2025-06-04', 5);
80
81 -- ----- 2.3 เพิ่มข้อมูลแบบระบุเฉพาะบางคอลัมน์ -----
82 -- คอลัมน์ที่ไม่ได้ระบุจะได้ค่า DEFAULT หรือ NULL
83 INSERT INTO students (student_code, first_name, last_name, age)
84 VALUES ('6712009', 'ทดสอบ', 'ระบบ', 19);
85
86
87 -- =====
88 -- ส่วนที่ 3 : แสดงข้อมูลด้วยคำสั่ง SELECT
89 -- =====
90
91 -- 3.1 แสดงรายชื่อนักศึกษาทั้งหมด <<< โจทย์ข้อ 1
92 SELECT * FROM students;
93
94 -- 3.2 แสดงเฉพาะคอลัมน์ที่ต้องการ พร้อมตั้งชื่อหัวตารางใหม่ด้วย AS
95 SELECT student_code AS 'รหัสนักศึกษา',
96 first_name AS 'ชื่อ',
97 last_name AS 'นามสกุล',
98 age AS 'อายุ'
99 FROM students;
100
101 -- 3.3 แสดงนักศึกษาที่มีอายุมากกว่า 20 ปี <<< โจทย์ข้อ 2
102 SELECT student_code, first_name, last_name, age
103 FROM students
104 WHERE age > 20;
105
106 -- 3.4 เรียงลำดับข้อมูลตามอายุจากมากไปน้อย
107 SELECT student_code, first_name, last_name, age
108 FROM students
109 ORDER BY age DESC;
110
111 -- 3.5 เงื่อนไขหลายข้อด้วย AND (นักศึกษาหญิง และ GPA ตั้งแต่ 3.00 ขึ้นไป)
112 SELECT student_code, first_name, last_name, gender, gpa
113 FROM students
114 WHERE gender = 'หญิง' AND gpa >= 3.00;
115
116 -- 3.6 ค้นหาข้อความบางส่วนด้วย LIKE (ชื่อขึ้นต้นด้วย "สม")
117 SELECT student_code, first_name, last_name
118 FROM students
119 WHERE first_name LIKE 'สม%';
120
121 -- 3.7 ค้นหาช่วงข้อมูลด้วย BETWEEN (อายุ 19 ถึง 21 ปี)
122 SELECT student_code, first_name, age
123 FROM students
124 WHERE age BETWEEN 19 AND 21;
125
126 -- 3.8 ฟังก์ชันสรุปผล (นับจำนวน / ค่าเฉลี่ย / มากสุด / น้อยสุด)
127 SELECT COUNT(*) AS 'จำนวนนักศึกษาทั้งหมด',
128 ROUND(AVG(age), 1) AS 'อายุเฉลี่ย',
129 MAX(age) AS 'อายุมากที่สุด',
130 MIN(age) AS 'อายุน้อยสุด',
131 ROUND(AVG(gpa), 2) AS 'GPA เฉลี่ย'

```

```

132 FROM students;
133
134 -- 3.9 จัดกลุ่มข้อมูลด้วย GROUP BY (นับจำนวนนักศึกษาแยกตามเพศ)
135 SELECT gender AS 'เพศ', COUNT(*) AS 'จำนวน'
136 FROM students
137 WHERE gender IS NOT NULL
138 GROUP BY gender;
139
140 -- 3.10 เชื่อม 2 ตารางด้วย INNER JOIN (นักศึกษา + วิชาที่ลงทะเบียน)
141 SELECT s.student_code AS 'รหัสนักศึกษา',
142        CONCAT(s.first_name, ' ', s.last_name) AS 'ชื่อ-นามสกุล',
143        c.course_code AS 'รหัสวิชา',
144        c.course_name AS 'ชื่อวิชา',
145        c.instructor AS 'ผู้สอน'
146 FROM students s
147 INNER JOIN courses c ON s.course_id = c.course_id
148 ORDER BY c.course_code;
149
150 -- 3.11 นับจำนวนนักศึกษาในแต่ละวิชา (LEFT JOIN + GROUP BY + HAVING)
151 SELECT c.course_code AS 'รหัสวิชา',
152        c.course_name AS 'ชื่อวิชา',
153        COUNT(s.student_id) AS 'จำนวนนักศึกษา'
154 FROM courses c
155 LEFT JOIN students s ON c.course_id = s.course_id
156 GROUP BY c.course_id, c.course_code, c.course_name
157 HAVING COUNT(s.student_id) > 0;
158
159 -- 3.12 แสดงข้อมูลรายวิชาทั้งหมด
160 SELECT * FROM courses;
161
162
163 -- =====
164 -- ส่วนที่ 4 : แก้ไขข้อมูลด้วยคำสั่ง UPDATE
165 -- =====
166
167 -- 4.1 แก้ไขชื่อ-นามสกุลของนักศึกษารหัส 6712003 <<< โจทย์ข้อ 3
168 UPDATE students
169 SET first_name = 'ปีติศักดิ์',
170     last_name = 'ยืนติมาก'
171 WHERE student_code = '6712003';
172
173 -- ตรวจสอบผลการแก้ไข
174 SELECT student_code, first_name, last_name
175 FROM students
176 WHERE student_code = '6712003';
177
178 -- 4.2 แก้ไขหลายคอลัมน์พร้อมกัน (อัปเดต GPA และอีเมล)
179 UPDATE students
180 SET gpa = 3.60,
181     email = 'weera.new@dusit.ac.th'
182 WHERE student_code = '6712005';
183
184 -- 4.3 แก้ไขโดยคำนวณจากค่าเดิม (เพิ่มอายุนักศึกษาทุกคน 1 ปี เมื่อขึ้นปีการศึกษาใหม่)
185 UPDATE students
186 SET age = age + 1
187 WHERE student_id > 0;
188
189 -- 4.4 แก้ไขแบบมีเงื่อนไข (นักศึกษาที่ยังไม่ได้ลงทะเบียนวิชา ให้ลงวิชา CS101)
190 UPDATE students
191 SET course_id = 1
192 WHERE course_id IS NULL;
193
194 -- ตรวจสอบผลหลังการแก้ไขทั้งหมด
195 SELECT student_code, first_name, last_name, age, gpa, course_id FROM students;
196
197
198 -- =====
199 -- ส่วนที่ 5 : ลบข้อมูลด้วยคำสั่ง DELETE
200 -- =====

```

```

201
202 -- 5.1 ลบข้อมุณักศึกษาที่กำหนด (รหัส 6712009 ซึ่งเป็นข้อมูลทดสอบ) <<< โจทย์ข้อ 4
203 DELETE FROM students
204 WHERE student_code = '6712009';
205
206 -- 5.2 ลบข้อมูลแบบมีเงื่อนไข (ลบนักศึกษาที่มีเกรดเฉลี่ยต่ำกว่า 3.00 ออกจากรายชื่อ)
207 DELETE FROM students
208 WHERE gpa < 3.00;
209
210 -- ตรวจสอบผลหลังการลบ
211 SELECT student_code, first_name, last_name, gpa FROM students;
212
213 -- 5.3 ลบรายวิชาที่ไม่มีนักศึกษาลงทะเบียน (ใช้ Subquery)
214 DELETE FROM courses
215 WHERE course_id NOT IN (
216     SELECT course_id FROM (
217         SELECT DISTINCT course_id FROM students WHERE course_id IS NOT NULL
218     ) AS t
219 );
220
221 SELECT * FROM courses;
222
223 -- *** คำเตือน : คำสั่งด้านล่างนี้จะลบข้อมูล "ทั้งหมด" จึงปิดคอมเมนต์ไว้ ***
224 -- DELETE FROM students;      -- ลบทุกแถว แต่โครงสร้างตารางยังอยู่
225 -- TRUNCATE TABLE students;  -- ลบทุกแถวและรีเซ็ตเลข AUTO_INCREMENT
226 -- DROP TABLE students;      -- ลบทั้งตาราง (โครงสร้างหายไปด้วย)
227
228
229 -- =====
230 -- จบในงานสัปดาห์ที่ 4
231 -- =====

```

ส่วนที่ 2 ผลลัพธ์การทำงานของคำสั่ง SQL

รันคำสั่งทุกบรรทัดจริงตามลำดับในสคริปต์ และบันทึกผลลัพธ์ที่ได้ไว้ที่ละคำสั่ง

0.1 เปิดการตรวจสอบคีย์นอก (Foreign Key) ของ SQLite

```
PRAGMA foreign_keys = ON;
```

Query OK, 0 row(s) affected

0.2 ลบตารางเดิมทิ้งก่อน (ถ้ามี) เพื่อให้รันซ้ำได้

```
DROP TABLE IF EXISTS students;
```

Query OK, 0 row(s) affected

0.3 ลบตาราง courses เดิมทิ้งก่อน (ถ้ามี)

```
DROP TABLE IF EXISTS courses;
```

Query OK, 0 row(s) affected

1.1 สร้างตาราง courses (รายวิชา)

```
CREATE TABLE courses (  
  course_id    INTEGER      PRIMARY KEY AUTOINCREMENT,  
  course_code  VARCHAR(10)  NOT NULL UNIQUE,  
  course_name  VARCHAR(100) NOT NULL,  
  credits      INTEGER      NOT NULL DEFAULT 3,  
  instructor   VARCHAR(60),  
  semester     VARCHAR(10),  
  CHECK (credits BETWEEN 1 AND 6)  
);
```

Query OK, 0 row(s) affected

1.2 สร้างตาราง students (นักศึกษา) พร้อมคีย์นอกอ้างอิงไปยัง courses

```
CREATE TABLE students (  
  student_id   INTEGER      PRIMARY KEY AUTOINCREMENT,  
  student_code VARCHAR(12)  NOT NULL UNIQUE,  
  first_name   VARCHAR(50)  NOT NULL,  
  last_name    VARCHAR(50)  NOT NULL,  
  gender       VARCHAR(10),  
  age          INTEGER      NOT NULL,  
  email        VARCHAR(80)  UNIQUE,  
  gpa          DECIMAL(3,2) DEFAULT 0.00,  
  enroll_date  DATE,  
  course_id    INTEGER,  
  CHECK (age >= 15),  
  FOREIGN KEY (course_id) REFERENCES courses(course_id)  
    ON UPDATE CASCADE ON DELETE SET NULL  
);
```

Query OK, 0 row(s) affected

2.1 เพิ่มข้อมูลรายวิชา 5 วิชา (เพิ่มหลายแถวในคำสั่งเดียว)

```
INSERT INTO courses (course_code, course_name, credits, instructor, semester) VALUES
('CS101', 'Introduction to Programming', 3, 'อ.สมชาย ใจดี', '1/2568'),
('CS102', 'Database Systems', 3, 'อ.สมหญิง รักเรียน', '1/2568'),
('CS201', 'Web Development', 3, 'อ.วิชัย เก่งกาจ', '1/2568'),
('MA101', 'Mathematics for Computing', 3, 'อ.มานี มีวิชา', '1/2568'),
('EN101', 'English for IT', 2, 'อ.John Smith', '1/2568');
```

Query OK, 5 row(s) affected

2.2 เพิ่มข้อมูลนักศึกษา 8 คน

```
INSERT INTO students
(student_code, first_name, last_name, gender, age, email, gpa, enroll_date, course_id)
VALUES
('6712001', 'สมชาย', 'ใจงาม', 'ชาย', 20, 'somchai@dusit.ac.th', 3.25, '2025-06-01', 1),
('6712002', 'สมหญิง', 'รักดี', 'หญิง', 19, 'somying@dusit.ac.th', 3.75, '2025-06-01', 2),
('6712003', 'ปิติ', 'ยินดี', 'ชาย', 22, 'piti@dusit.ac.th', 2.85, '2025-06-01', 1),
('6712004', 'มานี', 'มีนา', 'หญิง', 21, 'manee@dusit.ac.th', 3.50, '2025-06-02', 3),
('6712005', 'วิระ', 'กล้าหาญ', 'ชาย', 23, 'weera@dusit.ac.th', 2.40, '2025-06-02', 2),
('6712006', 'ชูใจ', 'ใจดี', 'หญิง', 18, 'choojai@dusit.ac.th', 3.90, '2025-06-03', 4),
('6712007', 'อนุชา', 'พากเพียร', 'ชาย', 20, 'anucha@dusit.ac.th', 3.10, '2025-06-03', 3),
('6712008', 'กาญจนา', 'ศรีสุข', 'หญิง', 24, 'kanjana@dusit.ac.th', 2.95, '2025-06-04', 5);
```

Query OK, 8 row(s) affected

2.3 เพิ่มข้อมูลแบบระบุเฉพาะบางคอลัมน์ (คอลัมน์ที่เหลือได้ค่า DEFAULT หรือ NULL)

```
INSERT INTO students (student_code, first_name, last_name, age)
VALUES ('6712009', 'ทดสอบ', 'ระบบ', 19);
```

Query OK, 1 row(s) affected

3.1 แสดงรายชื่อนักศึกษาทั้งหมด

```
SELECT * FROM students;
```

student_id	student_code	first_name	last_name	gender	age	email	gpa	enroll_date	course_id
1	6712001	สมชาย	ใจงาม	ชาย	20	somchai@dusit.ac.th	3.25	2025-06-01	1
2	6712002	สมหญิง	รักดี	หญิง	19	somying@dusit.ac.th	3.75	2025-06-01	2
3	6712003	ปิติ	ยินดี	ชาย	22	piti@dusit.ac.th	2.85	2025-06-01	1
4	6712004	มานี	มีนา	หญิง	21	manee@dusit.ac.th	3.50	2025-06-02	3
5	6712005	วิระ	กล้าหาญ	ชาย	23	weera@dusit.ac.th	2.40	2025-06-02	2
6	6712006	ชูใจ	ใจดี	หญิง	18	choojai@dusit.ac.th	3.90	2025-06-03	4
7	6712007	อนุชา	พากเพียร	ชาย	20	anucha@dusit.ac.th	3.10	2025-06-03	3
8	6712008	กาญจนา	ศรีสุข	หญิง	24	kanjana@dusit.ac.th	2.95	2025-06-04	5
9	6712009	ทดสอบ	ระบบ	NULL	19	NULL	0.00	NULL	NULL

9 row(s) in set

3.2 แสดงเฉพาะคอลัมน์ที่ต้องการ พร้อมตั้งชื่อหัวตารางใหม่ด้วย AS

```
SELECT student_code AS "รหัสนักศึกษา",  
       first_name  AS "ชื่อ",  
       last_name   AS "นามสกุล",  
       age         AS "อายุ"  
FROM students;
```

รหัสนักศึกษา	ชื่อ	นามสกุล	อายุ
6712001	สมชาย	ใจงาม	20
6712002	สมหญิง	รักดี	19
6712003	ปิติ	ยินดี	22
6712004	มานี	มีนา	21
6712005	วิระ	กล้าหาญ	23
6712006	ซูใจ	ใจดี	18
6712007	อนุชา	พากเพียร	20
6712008	กาญจนา	ศรีสุข	24
6712009	ทดสอบ	ระบบ	19

9 row(s) in set

3.3 แสดงนักศึกษาที่มีอายุมากกว่า 20 ปี

```
SELECT student_code, first_name, last_name, age  
FROM students  
WHERE age > 20;
```

student_code	first_name	last_name	age
6712003	ปิติ	ยินดี	22
6712004	มานี	มีนา	21
6712005	วิระ	กล้าหาญ	23
6712008	กาญจนา	ศรีสุข	24

4 row(s) in set

3.4 เรียงลำดับข้อมูลตามอายุจากมากไปน้อย

```
SELECT student_code, first_name, last_name, age
FROM students
ORDER BY age DESC;
```

student_code	first_name	last_name	age
6712008	กาญจนา	ศรีสุข	24
6712005	วีระ	กล้าหาญ	23
6712003	ปิติ	อินดี	22
6712004	มานี	มีนา	21
6712001	สมชาย	ใจงาม	20
6712007	อนุชา	พากเพียร	20
6712002	สมหญิง	รักดี	19
6712009	ทดสอบ	ระบบ	19
6712006	ซูใจ	ใจดี	18

9 row(s) in set

3.5 เงื่อนไขหลายข้อด้วย AND

```
SELECT student_code, first_name, last_name, gender, gpa
FROM students
WHERE gender = 'หญิง' AND gpa >= 3.00;
```

student_code	first_name	last_name	gender	gpa
6712002	สมหญิง	รักดี	หญิง	3.75
6712004	มานี	มีนา	หญิง	3.50
6712006	ซูใจ	ใจดี	หญิง	3.90

3 row(s) in set

3.6 ค้นหาข้อความบางส่วนด้วย LIKE

```
SELECT student_code, first_name, last_name
FROM students
WHERE first_name LIKE 'สม%';
```

student_code	first_name	last_name
6712001	สมชาย	ใจงาม
6712002	สมหญิง	รักดี

2 row(s) in set

3.7 ค้นหาช่วงข้อมูลด้วย BETWEEN

```
SELECT student_code, first_name, age
FROM students
WHERE age BETWEEN 19 AND 21;
```

student_code	first_name	age
6712001	สมชาย	20
6712002	สมหญิง	19
6712004	มานี	21
6712007	อนุชา	20
6712009	ทดสอบ	19

5 row(s) in set

3.8 ฟังก์ชันสรุปผล

```
SELECT COUNT(*) AS "จำนวนนักศึกษาทั้งหมด",
       ROUND(AVG(age), 1) AS "อายุเฉลี่ย",
       MAX(age) AS "อายุมากที่สุด",
       MIN(age) AS "อายุน้อยสุด",
       ROUND(AVG(gpa), 2) AS "GPA เฉลี่ย"
FROM students;
```

จำนวนนักศึกษาทั้งหมด	อายุเฉลี่ย	อายุมากที่สุด	อายุน้อยสุด	GPA เฉลี่ย
9	20.7	24	18	2.86

1 row(s) in set

3.9 จัดกลุ่มข้อมูลด้วย GROUP BY

```
SELECT gender AS "เพศ", COUNT(*) AS "จำนวน"
FROM students
WHERE gender IS NOT NULL
GROUP BY gender;
```

เพศ	จำนวน
ชาย	4
หญิง	4

2 row(s) in set

3.10 เชื่อม 2 ตารางด้วย INNER JOIN

```
SELECT s.student_code AS "รหัสนักศึกษา",
       s.first_name || ' ' || s.last_name AS "ชื่อ-นามสกุล",
       c.course_code AS "รหัสวิชา",
       c.course_name AS "ชื่อวิชา",
       c.instructor AS "ผู้สอน"
FROM students s
INNER JOIN courses c ON s.course_id = c.course_id
ORDER BY c.course_code;
```

รหัสนักศึกษา	ชื่อ-นามสกุล	รหัสวิชา	ชื่อวิชา	ผู้สอน
6712001	สมชาย ใจงาม	CS101	Introduction to Programming	อ.สมชาย ใจดี
6712003	ปิติ ยินดี	CS101	Introduction to Programming	อ.สมชาย ใจดี
6712002	สมหญิง รักดี	CS102	Database Systems	อ.สมหญิง รักเรียน
6712005	วิระ กล้าหาญ	CS102	Database Systems	อ.สมหญิง รักเรียน
6712004	มานี มีนา	CS201	Web Development	อ.วิชัย เก่งกาจ
6712007	อนุชา พากเพียร	CS201	Web Development	อ.วิชัย เก่งกาจ
6712008	กาญจนา ศรีสุข	EN101	English for IT	อ.John Smith
6712006	ซูโจ ใจดี	MA101	Mathematics for Computing	อ.มานี มีวิชา

8 row(s) in set

3.11 นับจำนวนนักศึกษาในแต่ละวิชา

```
SELECT c.course_code AS "รหัสวิชา",
       c.course_name AS "ชื่อวิชา",
       COUNT(s.student_id) AS "จำนวนนักศึกษา"
FROM courses c
LEFT JOIN students s ON c.course_id = s.course_id
GROUP BY c.course_id, c.course_code, c.course_name
HAVING COUNT(s.student_id) > 0;
```

รหัสวิชา	ชื่อวิชา	จำนวนนักศึกษา
CS101	Introduction to Programming	2
CS102	Database Systems	2
CS201	Web Development	2
EN101	English for IT	1
MA101	Mathematics for Computing	1

5 row(s) in set

3.12 แสดงข้อมูลรายวิชาทั้งหมด

```
SELECT * FROM courses;
```

course_id	course_code	course_name	credits	instructor	semester
1	CS101	Introduction to Programming	3	อ.สมชาย ใจดี	1/2568
2	CS102	Database Systems	3	อ.สมหญิง รักเรียน	1/2568
3	CS201	Web Development	3	อ.วิชัย เก่งกาจ	1/2568
4	MA101	Mathematics for Computing	3	อ.มานี มีวิชา	1/2568
5	EN101	English for IT	2	อ.John Smith	1/2568

5 row(s) in set

4.1 แก้ไขชื่อ-นามสกุลของนักศึกษารหัส 6712003

```
UPDATE students
SET first_name = 'ปิตัสศักดิ์',
    last_name = 'อินดีมก'
WHERE student_code = '6712003';
```

Query OK, 1 row(s) affected

ตรวจสอบผลการแก้ไขข้อ 4.1

```
SELECT student_code, first_name, last_name
FROM students
WHERE student_code = '6712003';
```

student_code	first_name	last_name
6712003	ปิตัสศักดิ์	อินดีมก

1 row(s) in set

4.2 แก้ไขหลายคอลัมน์พร้อมกัน

```
UPDATE students
SET gpa = 3.60,
    email = 'weera.new@dusit.ac.th'
WHERE student_code = '6712005';
```

Query OK, 1 row(s) affected

4.3 แก้ไขโดยคำนวณจากค่าเดิม (เพิ่มอายุทุกคน 1 ปี)

```
UPDATE students
SET age = age + 1
WHERE student_id > 0;
```

Query OK, 9 row(s) affected

4.4 แก้ไขแบบมีเงื่อนไข (คนที่ยังไม่มีวิชา ให้ลงวิชา CS101)

```
UPDATE students
SET course_id = 1
WHERE course_id IS NULL;
```

Query OK, 1 row(s) affected

ตรวจสอบผลหลังการ UPDATE ทั้งหมด

```
SELECT student_code, first_name, last_name, age, gpa, course_id FROM students;
```

student_code	first_name	last_name	age	gpa	course_id
6712001	สมชาย	ใจงาม	21	3.25	1
6712002	สมหญิง	รักดี	20	3.75	2
6712003	ปิติศักดิ์	ยินดีมาก	23	2.85	1
6712004	มานี	มีนา	22	3.50	3
6712005	วีระ	กล้าหาญ	24	3.60	2
6712006	ซูใจ	ใจดี	19	3.90	4
6712007	อนุชา	พากเพียร	21	3.10	3
6712008	กาญจนา	ศรีสุข	25	2.95	5
6712009	ทดสอบ	ระบบ	20	0.00	1

9 row(s) in set

5.1 ลบข้อมูลนักศึกษาที่กำหนด

```
DELETE FROM students
WHERE student_code = '6712009';
```

Query OK, 1 row(s) affected

5.2 ลบข้อมูลแบบมีเงื่อนไข (ลบนักศึกษาที่มีเกรดเฉลี่ยต่ำกว่า 3.00)

```
DELETE FROM students
WHERE gpa < 3.00;
```

Query OK, 2 row(s) affected

ตรวจสอบผลหลังการ DELETE

```
SELECT student_code, first_name, last_name, gpa FROM students;
```

student_code	first_name	last_name	gpa
6712001	สมชาย	ใจงาม	3.25
6712002	สมหญิง	รักดี	3.75
6712004	มานี	มีนา	3.50
6712005	วิระ	กล้าหาญ	3.60
6712006	ซูใจ	ใจดี	3.90
6712007	อนุชา	พากเพียร	3.10

6 row(s) in set

5.3 ลบรายวิชาที่ไม่มีนักศึกษาลงทะเบียน

```
DELETE FROM courses  
WHERE course_id NOT IN (SELECT DISTINCT course_id FROM students WHERE course_id IS NOT NULL);
```

Query OK, 1 row(s) affected

ตรวจสอบรายวิชาที่เหลืออยู่

```
SELECT * FROM courses;
```

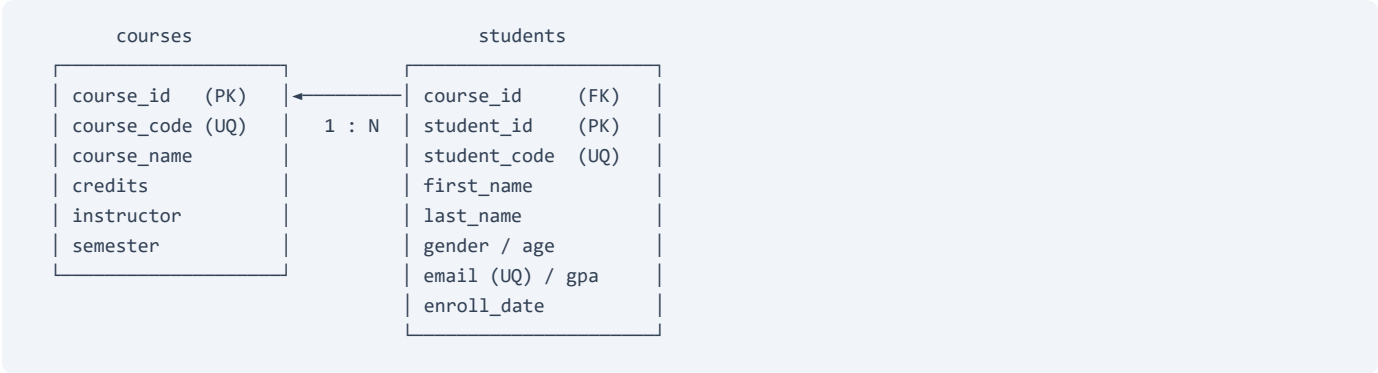
course_id	course_code	course_name	credits	instructor	semester
1	CS101	Introduction to Programming	3	อ.สมชาย ใจดี	1/2568
2	CS102	Database Systems	3	อ.สมหญิง รักเรียน	1/2568
3	CS201	Web Development	3	อ.วิชัย เก่งกาจ	1/2568
4	MA101	Mathematics for Computing	3	อ.มานี มีวิชา	1/2568

4 row(s) in set

ส่วนที่ 3 อธิบายคำสั่ง SQL ที่ใช้

1. โครงสร้างฐานข้อมูล

ฐานข้อมูล `student_management` ประกอบด้วย 2 ตาราง และเชื่อมกันด้วยคีย์นอก (Foreign Key)



ความสัมพันธ์: 1 รายวิชา มีนักศึกษาลงทะเบียนได้หลายคน (One-to-Many)

สัญลักษณ์	ความหมาย
PK (Primary Key)	คีย์หลัก ใช้ระบุแถวแบบไม่ซ้ำกัน ห้ามเป็นค่าว่าง
FK (Foreign Key)	คีย์นอก ใช้อ้างอิงไปยังคีย์หลักของอีกตารางหนึ่ง
UQ (Unique)	ค่าในคอลัมน์นี้ห้ามซ้ำกัน

2. คำสั่ง CREATE TABLE — สร้างตาราง

รูปแบบคำสั่ง

```
CREATE TABLE ชื่อตาราง (  
    ชื่อคอลัมน์ ชนิดข้อมูล [เงื่อนไขบังคับ],  
    ...  
);
```

ตัวอย่างจริงในใบงาน

```
CREATE TABLE courses (  
    course_id INT AUTO_INCREMENT PRIMARY KEY,  
    course_code VARCHAR(10) NOT NULL UNIQUE,  
    course_name VARCHAR(100) NOT NULL,  
    credits INT NOT NULL DEFAULT 3,  
    instructor VARCHAR(60),  
    semester VARCHAR(10),  
    CONSTRAINT chk_credits CHECK (credits BETWEEN 1 AND 6)  
)  
ENGINE=InnoDB;
```

อธิบายทีละส่วน

ส่วนของคำสั่ง	หน้าที่
CREATE TABLE courses	สร้างตารางชื่อ courses
INT	ชนิดข้อมูลจำนวนเต็ม
VARCHAR(50)	ข้อความความยาวไม่เกิน 50 ตัวอักษร (ใช้เนื้อที่เท่าที่เก็บจริง)
DECIMAL(3,2)	ตัวเลขทศนิยม รวม 3 หลัก ทศนิยม 2 ตำแหน่ง เช่น 3.25 เหมาะกับ GPA
DATE	เก็บวันที่ รูปแบบ YYYY-MM-DD
PRIMARY KEY	กำหนดให้เป็นคีย์หลัก ห้ามซ้ำและห้ามว่าง
AUTO_INCREMENT	ให้ระบบใส่เลขลำดับให้อัตโนมัติ (1, 2, 3, ...) ไม่ต้องกรอกเอง
NOT NULL	คอลัมน์นี้ต้องมีค่าเสมอ ปกติว่างไม่ได้
UNIQUE	ค่าในคอลัมน์นี้ห้ามซ้ำกับแถวอื่น เช่น รหัสนักศึกษา อีเมล
DEFAULT 3	ถ้าไม่ระบุค่าตอน INSERT ระบบจะใส่ค่า 3 ให้
CHECK (...)	ตรวจสอบเงื่อนไขก่อนบันทึก ถ้าผิดเงื่อนไขจะบันทึกไม่ได้
ENGINE=InnoDB	เลือกเครื่องมือจัดเก็บของ MySQL ที่รองรับ Foreign Key

การประกาศคีย์นอก (Foreign Key)

```
CONSTRAINT fk_student_course FOREIGN KEY (course_id)
REFERENCES courses(course_id)
ON UPDATE CASCADE
ON DELETE SET NULL
```

- FOREIGN KEY (course_id) REFERENCES courses(course_id) — คอลัมน์ course_id ในตาราง students ต้องมีค่าตรงกับ course_id ที่มีอยู่จริงในตาราง courses เท่านั้น (ป้องกันการกรอกรหัสวิชาที่ไม่มีอยู่)
- ON UPDATE CASCADE — ถ้ารหัสวิชาต้นทางถูกแก้ไข ค่าที่อ้างอิงอยู่ในตาราง students จะถูกแก้ตามให้อัตโนมัติ
- ON DELETE SET NULL — ถ้ารายวิชาถูกลบ ค่า course_id ของนักศึกษาที่เคยลงวิชานั้นจะกลายเป็น NULL (แทนที่จะลบนักศึกษาทิ้งไปด้วย)

ข้อควรจำ: ต้องสร้างตาราง courses ก่อน students เสมอ เพราะตาราง students อ้างอิงไปหา courses ถ้าสร้างสลับกันจะเกิด error

3. คำสั่ง INSERT — เพิ่มข้อมูล

รูปแบบคำสั่ง

```
INSERT INTO ชื่อตาราง (คอลัมน์1, คอลัมน์2, ...)
VALUES (ค่า1, ค่า2, ...);
```

แบบที่ 1 : เพิ่มหลายแถวในคำสั่งเดียว

```
INSERT INTO courses (course_code, course_name, credits, instructor, semester) VALUES
('CS101', 'Introduction to Programming', 3, 'อ.สมชาย ใจดี', '1/2568'),
('CS102', 'Database Systems', 3, 'อ.สมหญิง รักเรียน', '1/2568');
```

คั่นแต่ละแถวด้วยเครื่องหมายจุลภาค , แล้วปิดท้ายคำสั่งด้วย ; เพียงครั้งเดียว — วิธีนี้ทำงานเร็วกว่าการเขียน INSERT แยกหลายคำสั่ง

แบบที่ 2 : ระบุเฉพาะบางคอลัมน์

```
INSERT INTO students (student_code, first_name, last_name, age)
VALUES ('6712009', 'ทดสอบ', 'ระบบ', 19);
```

คอลัมน์ที่ไม่ได้ระบุจะได้ค่า DEFAULT (เช่น gpa = 0.00) หรือ NULL (เช่น email, enroll_date)

หลักการที่ต้องจำ

- ข้อมูลที่เป็น ข้อความและวันที่ ต้องครอบด้วยเครื่องหมายคำพูดเดี่ยว '...' — ส่วน ตัวเลข ไม่ต้องครอบ
- ลำดับของค่าใน VALUES ต้องเรียงตรงกับลำดับชื่อคอลัมน์ที่ระบุไว้ข้างหน้า
- ไม่ต้องใส่ค่าให้คอลัมน์ที่เป็น AUTO_INCREMENT เพราะระบบสร้างเลขให้เอง
- ถ้าใส่ค่าซ้ำในคอลัมน์ UNIQUE หรือใส่ค่าที่ผิดเงื่อนไข CHECK คำสั่งจะล้มเหลวทั้งหมด

4. คำสั่ง SELECT — แสดงข้อมูล

รูปแบบคำสั่งเต็ม (ลำดับการเขียนห้ามสลับ)

```
SELECT คอลัมน์ที่ต้องการ
FROM ชื่อตาราง
WHERE เงื่อนไขการกรองแถว
GROUP BY คอลัมน์ที่ใช้จัดกลุ่ม
HAVING เงื่อนไขการกรองกลุ่ม
ORDER BY คอลัมน์ที่ใช้เรียงลำดับ;
```

4.1 แสดงข้อมูลทั้งหมด

```
SELECT * FROM students;
```

เครื่องหมาย * หมายถึง "ทุกคอลัมน์" — ผลลัพธ์ที่ได้คือนักศึกษาทั้ง 9 คน พร้อมทุกคอลัมน์

4.2 เลือกเฉพาะคอลัมน์ + เปลี่ยนชื่อหัวตารางด้วย AS

```
SELECT student_code AS 'รหัสนักศึกษา',
       first_name AS 'ชื่อ',
       age AS 'อายุ'
FROM students;
```

AS ใช้ตั้งชื่อเรียกใหม่ (Alias) ให้หัวตารางอ่านง่ายขึ้น ไม่ได้เปลี่ยนชื่อคอลัมน์จริงในฐานข้อมูล

4.3 กรองข้อมูลด้วย WHERE

```
SELECT student_code, first_name, last_name, age
FROM students
WHERE age > 20;
```

WHERE คือการตั้งเงื่อนไขว่าจะเอาแถวไหนบ้าง — ตัวอย่างนี้ได้เฉพาะนักศึกษาที่อายุมากกว่า 20 ปี (ผลลัพธ์ 4 คน)

ตัวดำเนินการเปรียบเทียบที่ใช้บ่อย

ตัวดำเนินการ	ความหมาย	ตัวอย่าง
=	เท่ากับ	WHERE gender = 'หญิง'
> <	มากกว่า / น้อยกว่า	WHERE age > 20
>= <=	มากกว่าหรือเท่ากับ / น้อยกว่าหรือเท่ากับ	WHERE gpa >= 3.00
<> หรือ !=	ไม่เท่ากับ	WHERE gender <> 'ชาย'
AND	ต้องเป็นจริงทุกเงื่อนไข	WHERE gender = 'หญิง' AND gpa >= 3.00
OR	เป็นจริงข้อใดข้อหนึ่งก็พอ	WHERE age < 19 OR age > 23
BETWEEN a AND b	อยู่ในช่วง a ถึง b (รวมค่าปลายทั้งสอง)	WHERE age BETWEEN 19 AND 21
IN (...)	มีค่าตรงกับค่าใดค่าหนึ่งในวงเล็บ	WHERE course_id IN (1, 2)
LIKE	ค้นหาข้อความบางส่วน	WHERE first_name LIKE 'สม%'
IS NULL / IS NOT NULL	ไม่มีค่า / มีค่า	WHERE email IS NULL

สัญลักษณ์ที่ใช้กับ LIKE

- 'สม%' = ขึ้นต้นด้วย "สม"
- '%ใจ' = ลงท้ายด้วย "ใจ"
- '%ชา%' = มีคำว่า "ชา" อยู่ตรงไหนก็ได้
- _ = แทนตัวอักษรใดก็ได้ 1 ตัว

ข้อควรระวัง: การหาค่าว่างต้องใช้ IS NULL เท่านั้น เขียน = NULL จะไม่ได้ผลลัพธ์เพราะ NULL แปลว่า "ไม่ทราบค่า" จึงเปรียบเทียบเท่ากันไม่ได้

4.4 เรียงลำดับด้วย ORDER BY

```
SELECT student_code, first_name, age
FROM students
ORDER BY age DESC;
```

- ASC = น้อยไปมาก (ค่าเริ่มต้น ไม่เขียนก็ได้)
- DESC = มากไปน้อย
- เรียงหลายชั้นได้ เช่น ORDER BY age DESC, first_name ASC

4.5 ฟังก์ชันสรุปผล (Aggregate Function)

```
SELECT COUNT(*) AS 'จำนวนนักศึกษาทั้งหมด',
       ROUND(AVG(age), 1) AS 'อายุเฉลี่ย',
       MAX(age) AS 'อายุมากที่สุด',
       MIN(age) AS 'อายุน้อยสุด',
       ROUND(AVG(gpa), 2) AS 'GPA เฉลี่ย'
FROM students;
```

ฟังก์ชัน	หน้าที่
COUNT(*)	นับจำนวนแถว
SUM(คอลัมน์)	หาผลรวม
AVG(คอลัมน์)	หาค่าเฉลี่ย
MAX() / MIN()	หาค่ามากที่สุด / น้อยสุด
ROUND(ค่า, n)	ปัดทศนิยมให้เหลือ n ตำแหน่ง

4.6 จัดกลุ่มด้วย GROUP BY

```
SELECT gender AS 'เพศ', COUNT(*) AS 'จำนวน'
FROM students
WHERE gender IS NOT NULL
GROUP BY gender;
```

GROUP BY ยุบแถวที่มีค่าเหมือนกันให้เหลือกลุ่มละ 1 แถว แล้วคำนวณสรุปในแต่ละกลุ่ม — ตัวอย่างนี้ได้ผลลัพธ์ 2 แถว คือ ชาย 4 คน และ หญิง 4 คน

4.7 เชื่อมตารางด้วย JOIN

```
SELECT s.student_code AS 'รหัสนักศึกษา',
       CONCAT(s.first_name, ' ', s.last_name) AS 'ชื่อ-นามสกุล',
       c.course_code AS 'รหัสวิชา',
       c.course_name AS 'ชื่อวิชา'
FROM students s
INNER JOIN courses c ON s.course_id = c.course_id
ORDER BY c.course_code;
```

- students s และ courses c — ตั้งชื่อย่อให้ตารางเพื่อพิมพ์สั้นลง
- ON s.course_id = c.course_id — เงื่อนไขการจับคู่แถวของสองตาราง
- CONCAT() — ต่อข้อความเข้าด้วยกัน (ใน SQLite ใช้เครื่องหมาย || แทน)

ชนิดของ JOIN	ผลลัพธ์ที่ได้
INNER JOIN	เอาเฉพาะแถวที่จับคู่กันได้ทั้งสองตาราง
LEFT JOIN	เอาทุกแถวของตารางซ้าย ถ้าจับคู่ไม่ได้ฝั่งขวาจะเป็น NULL
RIGHT JOIN	เอาทุกแถวของตารางขวา ถ้าจับคู่ไม่ได้ฝั่งซ้ายจะเป็น NULL

4.8 HAVING ต่างจาก WHERE อย่างไร

```
SELECT c.course_code, COUNT(s.student_id) AS 'จำนวนนักศึกษา'
FROM courses c
LEFT JOIN students s ON c.course_id = s.course_id
GROUP BY c.course_id, c.course_code
HAVING COUNT(s.student_id) > 0;
```

- WHERE กรอง แถวข้อมูลดิบ ก่อนจัดกลุ่ม → ใช้กับฟังก์ชันสรุปผลไม่ได้
- HAVING กรอง กลุ่มที่สรุปผลแล้ว → ใช้กับ COUNT(), SUM(), AVG() ได้

5. คำสั่ง UPDATE — แก้ไขข้อมูล

รูปแบบคำสั่ง

```
UPDATE ชื่อตาราง
SET คอลัมน์1 = ค่าใหม่1,
   คอลัมน์2 = ค่าใหม่2
WHERE เงื่อนไข;
```

5.1 แก้ไขชื่อนักศึกษา (ตามโจทย์)

```
UPDATE students
SET first_name = 'ปัดศักดิ์',
   last_name = 'ยืนติ่มมาก'
WHERE student_code = '6712003';
```

ผลลัพธ์: Query OK, 1 row affected — ข้อมูลของนักศึกษารหัส 6712003 เปลี่ยนจาก "ปติ ยินดี" เป็น "ปติศักดิ์ ยินดีมาก" ส่วนคนอื่นไม่ถูกแตะต้อง

5.2 แก้ไขหลายคอลัมน์พร้อมกัน

```
UPDATE students
SET gpa = 3.60,
    email = 'weera.new@dusit.ac.th'
WHERE student_code = '6712005';
```

ใช้ SET เพียงครั้งเดียว แล้วคั่นแต่ละคอลัมน์ด้วยจุลภาค ,

5.3 แก้ไขโดยคำนวณจากค่าเดิม

```
UPDATE students
SET age = age + 1
WHERE student_id > 0;
```

นำค่าเดิมในคอลัมน์มาบวกเพิ่มได้โดยตรง — ใช้ตอนขึ้นปีการศึกษาใหม่ นักศึกษาทุกคนอายุเพิ่ม 1 ปี

หมายเหตุ: โดยปกติควรเขียนเป็น `UPDATE students SET age = age + 1;` ใดๆ แต่ MySQL Workbench เปิดโหมด Safe Update ไว้ ซึ่งห้ามสั่ง UPDATE โดยไม่มี WHERE จึงใส่เงื่อนไข `WHERE student_id > 0` (ซึ่งเป็นจริงทุกแถว) เพื่อให้คำสั่งทำงานได้

⚠ ข้อควรระวังที่สำคัญที่สุด

ถ้าลืมใส่ WHERE ข้อมูลทุกแถวในตารางจะถูกแก้ทั้งหมด และย้อนกลับไม่ได้

```
UPDATE students SET first_name = 'สมชาย'; -- ผิด! ทุกคนจะชื่อสมชายหมด
```

วิธีป้องกัน: เขียน SELECT ด้วยเงื่อนไขเดียวกันดูก่อนเสมอว่าได้แถวที่ต้องการจริงหรือไม่ แล้วค่อยเปลี่ยนคำว่า SELECT เป็น UPDATE

6. คำสั่ง DELETE — ลบข้อมูล

รูปแบบคำสั่ง

```
DELETE FROM ชื่อตาราง
WHERE เงื่อนไข;
```

6.1 ลบนักศึกษาที่กำหนด (ตามโจทย์)

```
DELETE FROM students
WHERE student_code = '6712009';
```

ลบเฉพาะแถวที่รหัสนักศึกษาตรงกับ 6712009 → Query OK, 1 row affected

6.2 ลบแบบมีเงื่อนไข (ลบได้หลายแถวพร้อมกัน)

```
DELETE FROM students
WHERE gpa < 3.00;
```

ทุกแถวที่เข้าเงื่อนไขจะถูกลบพร้อมกัน → ในใบงานนี้ลบไป 2 คน (GPA 2.85 และ 2.95)

6.3 ลบโดยใช้ Subquery

```
DELETE FROM courses
WHERE course_id NOT IN (
```

```
SELECT DISTINCT course_id FROM students WHERE course_id IS NOT NULL
);
```

คำสั่งด้านในวงเล็บ (Subquery) จะทำงานก่อน เพื่อหารหัสวิชาที่ยังมีนักศึกษาลงทะเบียนอยู่ จากนั้นคำสั่งด้านนอกจึงลบวิชาที่ไม่อยู่ในรายการนั้น (วิชา EN101 ถูกลบเพราะนักศึกษาคนเดียวที่ลงวิชานี้ถูกลบไปในข้อ 6.2)

เปรียบเทียบคำสั่งลบ 3 แบบ

คำสั่ง	สิ่งที่ถูกลบ	โครงสร้างตาราง	ย้อนกลับได้ไหม
DELETE FROM students WHERE ...	เฉพาะแถวที่ตรงเงื่อนไข	ยังอยู่	ย้อนได้ถ้าใช้ TRANSACTION
TRUNCATE TABLE students	ทุกแถว + รีเซ็ตเลข AUTO_INCREMENT	ยังอยู่	ไม่ได้
DROP TABLE students	ทั้งตาราง	หายไปด้วย	ไม่ได้

ข้อควรระวัง: DELETE FROM students; โดยไม่ใส่ WHERE จะลบข้อมูลนักศึกษาทุกคนทันที

7. ตารางสรุปคำสั่ง SQL ที่ใช้ในใบงาน

ประเภท	คำสั่ง	หน้าที่	ตัวอย่างในใบงาน
DDL	CREATE DATABASE	สร้างฐานข้อมูล	CREATE DATABASE student_management;
DDL	CREATE TABLE	สร้างตาราง	สร้างตาราง students, courses
DDL	DROP TABLE	ลบตารางทั้งทั้งโครงสร้าง	DROP TABLE IF EXISTS students;
DML	INSERT	เพิ่มข้อมูลเข้าไปในตาราง	เพิ่มนักศึกษา 9 คน / รายวิชา 5 วิชา
DML	SELECT	ค้นหาและแสดงข้อมูล	แสดงรายชื่อ / นักศึกษาอายุเกิน 20 ปี
DML	UPDATE	แก้ไขข้อมูลเดิม	แก้ไขนักศึกษารหัส 6712003
DML	DELETE	ลบข้อมูลบางแถว	ลบนักศึกษารหัส 6712009

- DDL (Data Definition Language) = คำสั่งจัดการ *โครงสร้าง* ของฐานข้อมูล
- DML (Data Manipulation Language) = คำสั่งจัดการ *ข้อมูล* ที่อยู่ในตาราง

8. สรุปผลการทำงาน (ใช้เขียนลงในรายงาน)

ขั้นตอน	คำสั่งที่ใช้	ผลลัพธ์ที่ได้
สร้างฐานข้อมูลและตาราง	CREATE DATABASE / CREATE TABLE	ได้ 2 ตาราง คือ students และ courses
เพิ่มข้อมูล	INSERT	เพิ่มรายวิชา 5 วิชา และนักศึกษา 9 คน
แสดงรายชื่อนักศึกษาทั้งหมด	SELECT *	แสดงผล 9 แถว
แสดงนักศึกษาอายุมากกว่า 20 ปี	SELECT ... WHERE age > 20	แสดงผล 4 คน
แก้ไขชื่อนักศึกษา	UPDATE ... WHERE	แก้ไขสำเร็จ 1 แถว
เพิ่มอายุนักศึกษาทุกคน 1 ปี	UPDATE ... SET age = age + 1	แก้ไขสำเร็จ 9 แถว
ลบนักศึกษาที่กำหนด	DELETE ... WHERE student_code = '6712009'	ลบสำเร็จ 1 แถว
ลบนักศึกษาที่ GPA ต่ำกว่า 3.00	DELETE ... WHERE gpa < 3.00	ลบสำเร็จ 2 แถว เหลือนักศึกษา 6 คน

9. วิธีรันและ capture หน้าจอผลลัพธ์

แบบที่ 1 — ใช้ phpMyAdmin (XAMPP)

1. เปิด XAMPP Control Panel กด **Start** ที่ Apache และ MySQL
2. เปิดเบราว์เซอร์ไปที่ `http://localhost/phpmyadmin`
3. คลิกแท็บ **SQL** ด้านบน
4. คัดลอกคำสั่งจากไฟล์ `student_management_mysql.sql` ไปวาง (แนะนำให้อ่านที่ละส่วน ส่วนที่ 1 → 2 → 3 → 4 → 5 เพื่อให้ capture ผลลัพธ์แยกทีละข้อได้)
5. กดปุ่ม **Go** แล้วกด `Print Screen` หรือ `Win + Shift + S` เพื่อ capture หน้าจอผลลัพธ์

แบบที่ 2 — ใช้ MySQL Command Line

```
mysql -u root -p < student_management_mysql.sql
```

แบบที่ 3 — รันด้วย Python + SQLite (ไม่ต้องติดตั้ง MySQL)

```
python run_demo.py
```

โปรแกรมจะสร้างไฟล์ผลลัพธ์ให้ 3 ไฟล์

- `ผลลัพธ์.txt` — ผลลัพธ์แบบตัวอักษร เหมือนที่เห็นใน MySQL Command Line
- `results.html` — ผลลัพธ์แบบตาราง เปิดด้วยเบราว์เซอร์แล้ว capture ได้สวยงาม (แนะนำให้ใช้ไฟล์นี้ในการ capture)
- `student_management.db` — ไฟล์ฐานข้อมูล เปิดดูต่อได้ด้วยโปรแกรม DB Browser for SQLite

10. สิ่งที่ต้องส่ง (Checklist)

- ☐ **SQL Script** → ไฟล์ `student_management_mysql.sql`
- ☐ **Capture หน้าจอผลลัพธ์** → เปิด `results.html` แล้วถ่ายภาพหน้าจอ หรือ capture จากหน้าจอ phpMyAdmin
- ☐ **คำอธิบายคำสั่ง SQL** → เอกสารฉบับนี้ (อธิบายคำสั่ง-SQL.md)
- ☐ เขียนชื่อ-นามสกุล และรหัสนักศึกษาในหัวกระดาษของทุกไฟล์ก่อนส่ง